

Design Document: SCRMVF v2.0

Stochastic Credit Risk Modeling & Validation Framework

Poziom: Staff Engineer / Senior Research Scientist

Departament Ryzyka i Analiz Ilościowych

Kwiecień 2026

Streszczenie

Niniejszy dokument definiuje specyfikację implementacyjną systemu SCRMVF, przeznaczonego do symulacji ryzyka kredytowego w skali instytucjonalnej. Framework integruje zaawansowane modele stochastyczne typu *Structural Models of Default*, rygorystyczne potoki uczenia maszynowego (ML Pipeline) oraz wysokowydajne silniki symulacyjne oparte na wektoryzacji. System został zaprojektowany z myślą o zgodności z regulacjami IFRS 9 oraz Basel IV [1].

Spis treści

1	Core Modeling: Rozszerzony Model Vasicka	2
1.1	Dynamika Aktywów (Factor Structure)	2
1.2	Stochastyczny Parametr LGD	2
2	Machine Learning Pipeline: Kalibracja PD	2
2.1	Estymacja i Kalibracja	2
2.2	Handling Imbalanced Data	2
3	System Architecture & Data Engineering	2
3.1	Database Schema (SQLite/Analytical)	3
3.2	Diagram Systemu (TikZ Optimization)	3
4	Symulacja Monte Carlo: Optymalizacja	3
4.1	Vectorization and Memory Sharding	3
5	Zadania Implementacyjne	3
5.1	Task 1: Data Infrastructure & ETL	3
5.2	Task 2: Stochastic Engine (The Core)	4
5.3	Task 3: Risk Reporting & Validation	4

1 Core Modeling: Rozszerzony Model Vasicka

Podstawą systemu jest model Vasicka, jednak w wersji 2.0 przechodzimy na strukturę wieloczynnikową (*Multi-Factor Extension*), aby poprawnie modelować korelacje międzysektorowe [2].

1.1 Dynamika Aktywów (Factor Structure)

Kondycja finansowa $X_{i,s}$ kredytobiorcy i należącego do sektora k w scenariuszu s jest opisana równaniem:

$$X_{i,s} = w_i \left(\sum_{j=1}^m \beta_{k,j} Y_{j,s} \right) + \sqrt{1 - w_i^2} Z_{i,s} \quad (1)$$

Gdzie:

- $Y_{j,s} \sim \mathcal{N}(0, 1)$ – Czynniki systemowe (np. PKB, stopy procentowe, indeksy branżowe).
- $\beta_{k,j}$ – Ładunki czynnikowe określające wrażliwość sektora k na czynnik j .
- $Z_{i,s} \sim \mathcal{N}(0, 1)$ – Ryzyko idiosynkratyczne (specyficzne dla podmiotu).
- $w_i \in [0, 1]$ – Parametr determinujący *Asset Correlation* ($\rho = w_i^2$).

1.2 Stochastyczny Parametr LGD

W przeciwieństwie do modeli bazowych, parametr *Loss Given Default* (LGD) nie jest stały. Wprowadzamy rozkład Beta dla LGD, co pozwala na modelowanie niepewności odzysku:

$$LGD_{i,s} \sim \text{Beta}(\alpha, \beta), \quad \text{gdzie } \mu_{LGD} = \frac{\alpha}{\alpha + \beta} \quad (2)$$

Wymaga to od silnika symulacyjnego generowania dodatkowych $N \times S$ zmiennych losowych, co stawia wysokie wymagania przed generatorem PRNG (*Pseudo-Random Number Generator*).

2 Machine Learning Pipeline: Kalibracja PD

System SCRMVF implementuje pełny cykl życia modelu *Probability of Default* (PD).

2.1 Estymacja i Kalibracja

W fazie *Model Calibration*, system wykorzystuje dane historyczne do wytrenowania klasyfikatora (rekomendowany XGBoost lub Regresja Logistyczna z regularyzacją L_2). Kluczowym etapem jest *Probability Calibration* przy użyciu algorytmu **Isotonic Regression**, aby zapewnić, że prognozowane prawdopodobieństwa odpowiadają rzeczywistym częstościom bankructw [3].

2.2 Handling Imbalanced Data

Zbiory danych finansowych charakteryzują się wysokim *Class Imbalance*. Implementacja musi uwzględniać techniki takie jak *Scale_pos_weight* lub zaawansowane funkcje straty typu *Focal Loss* [4].

3 System Architecture & Data Engineering

Projekt musi realizować wzorzec *Hexagonal Architecture* (Ports and Adapters), izolując silnik matematyczny od infrastruktury [5].

3.1 Database Schema (SQLite/Analytical)

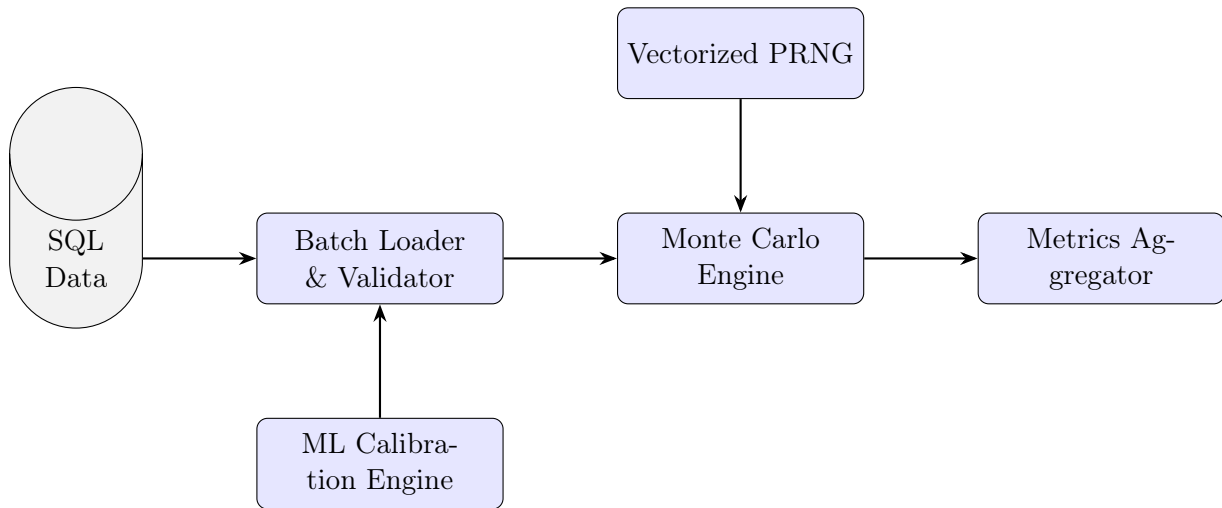
Warstwa *Persistence* musi obsługiwać efektywne zapytania analityczne. Wymagana jest implementacja indeksów na kolumnach `sector_id` oraz `rating_class`.

```
CREATE TABLE portfolio (  
    loan_id UUID PRIMARY KEY,  
    principal_amount INTEGER, -- Store in minor units (cents)  
    pd_point_in_time REAL,  
    lgd_mean REAL,  
    asset_correlation REAL,  
    sector_id INTEGER,  
    INDEX idx_sector (sector_id)  
);
```

Listing 1: Schemat SQL tabeli portfela

3.2 Diagram Systemu (TikZ Optimization)

Poniższy diagram przedstawia przepływ danych w systemie.



4 Symulacja Monte Carlo: Optymalizacja

Ze względu na rozmiar portfela (10^6 rekordów) i liczbę scenariuszy (10^5), silnik musi stosować techniki *High-Performance Computing* (HPC).

4.1 Vectorization and Memory Sharding

Obliczenia kondycji X muszą być wykonywane przy użyciu operacji macierzowych:

$$\mathbf{X} = \sqrt{\rho} \cdot (\mathbf{YB}^T) + \sqrt{1 - \rho} \cdot \mathbf{Z} \quad (3)$$

W przypadku ograniczeń pamięci RAM (VRAM), system musi implementować **Chunking Strategy**, dzieląc macierz $\mathbf{Z}$ na bloki przetwarzane sekwencyjnie przy zachowaniu reprodukowalności (zarządzanie *PRNG State*).

5 Zadania Implementacyjne

5.1 Task 1: Data Infrastructure & ETL

Zaimplementuj generator syntetycznych danych finansowych, który tworzy korelacje między cechami (np. wyższy LTV skorelowany z wyższym PD). Dane muszą zostać załadowane do SQLite.

5.2 Task 2: Stochastic Engine (The Core)

Stwórz moduł `MonteCarloSimulator`. Wykorzystaj `numpy.random.Generator` (PCG64) dla zapewnienia wysokiej jakości szumu. Zaimplementuj model wieloczynnikowy opisany w Sekcji 1.

5.3 Task 3: Risk Reporting & Validation

Wygeneruj raport walidacyjny zawierający:

- Wykres gęstości prawdopodobieństwa strat (*Loss PDF*).
- Obliczenie **Economic Capital** jako $VaR_{99,9\%} - \mathbb{E}[Loss]$.
- Testy stabilności: sprawdź błąd standardowy estymatora Monte Carlo.

Literatura

- [1] Basel Committee on Banking Supervision. (2017). *Basel III: Finalising post-crisis reforms*. Bank for International Settlements.
- [2] Vasicek, O. A. (2002). *The distribution of loan portfolio value*. Risk, 15(12), 160-162.
- [3] Platt, J. (1999). *Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Logistic Regression Methods*. Advances in Large Margin Classifiers.
- [4] Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). *Focal loss for dense object detection*. Proceedings of the IEEE international conference on computer vision.
- [5] Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.